

Episode Alerts

Mobile Application Project Report

Project Title: Episode Alerts Mobile App
Technology Stack: React Native, Expo, TypeScript
Prepared By: Nandu Krishna M 25CSE2015
Prepared For: Object Oriented Software Engineering Lab
Date: April 16, 2026

Contents

1	Introduction	1
1.1	Project Introduction	1
1.2	Objective	1
1.3	Problem Statement	1
2	Background (Related Work)	2
2.1	Background	2
2.2	Related Work	2
2.3	Project Positioning	2
3	SRS	3
3.1	Introduction	3
3.1.1	Purpose	3
3.1.2	Scope	3
3.1.3	Definitions, Acronyms, and Abbreviations	3
3.1.4	References	4
3.1.5	Overview	4
3.2	Overall Description	4
3.2.1	Product Perspective	4
3.2.2	Product Function	4
3.2.3	User Characteristics	5
3.2.4	General Constraints	5
3.2.5	Assumptions and Dependencies	5
3.3	Specific Requirements	5
3.3.1	External Interface Requirements	5
3.3.2	Functional Requirements	6
3.3.3	Performance Requirements	8
3.3.4	Design Constraints	8
3.3.5	Attributes	8
3.3.6	Other Attributes	8

4	Design	9
4.1	UML	9
4.1.1	Sequence Diagram	9
4.2	DFD	10
4.2.1	Level 0 Context DFD	10
4.2.2	Level 1 Logical DFD	10
4.2.3	Level 2 DFD: Content Discovery and Search	10
4.3	Use Cases	12
5	Code	17
5.1	Implementation Overview	17
5.1.1	TMDBService	17
5.1.2	WatchlistService	28
6	Results	39
6.1	Results Overview	39
6.2	Application Screenshots	40
6.3	Outcome Summary	42
6.4	Conclusion	43
	References	44

Chapter 1

Introduction

1.1 Project Introduction

Episode Alerts is a mobile application designed to help users discover television shows, organize personal watchlists, and stay informed about upcoming episodes. The project is implemented using React Native, Expo, and TypeScript, with TMDB as the primary source for show metadata and content discovery.

1.2 Objective

The objective of this project is to provide a practical, user-friendly companion app for TV tracking that combines discovery, watchlist management, reminders, and preferences in a single workflow. The implementation focuses on responsive interaction, modular service design, and reliable behavior in both online and cache-assisted scenarios.

1.3 Problem Statement

TV viewers often rely on scattered sources and manual reminders to keep track of shows, episode releases, and personal watch progress. This leads to missed episodes, inconsistent watchlist maintenance, and a fragmented user experience across discovery, details, and tracking tasks. The problem addressed by Episode Alerts is to provide a single, reliable mobile workflow that unifies show discovery, detailed show information, watchlist management, reminder scheduling, and preference handling, while remaining usable even when network availability is limited through cache-assisted behavior and optional cloud synchronization.

Chapter 2

Background (Related Work)

2.1 Background

TV tracking is fragmented when users have to move between search, release calendars, watchlists, and reminders. Episode Alerts addresses that by combining show discovery, show details, watchlist management, notifications, and cached fallback behavior in one mobile app.

2.2 Related Work

A recent App Store release, [CineSync Tracker](#), follows the same core direction: a unified movie and TV watchlist, a release calendar, search/discovery, and a minimalist interface.

The original Expo post by the app author, [this post](#), presents CineSync Tracker as a free minimalist alternative built with Expo and designed around utility.

The follow-up comment exchange, [this thread](#), confirms TMDB as the data source and notes the free-tier limit of about 40 requests every 10 seconds. That supports a cache-first design and fallback handling in Episode Alerts.

2.3 Project Positioning

Episode Alerts is TV-first, cache-aware, and focused on practical tracking rather than general catalog browsing.

Chapter 3

SRS

3.1 Introduction

3.1.1 Purpose

This software requirements specification describes the functional and non-functional requirements for Episode Alerts, a cross-platform mobile app that helps users discover television shows, manage a personal watchlist, and stay updated on upcoming episodes. It is intended for developers, testers, reviewers, and future maintainers who need a clear understanding of what the system should do and the limits it must work within.

3.1.2 Scope

Episode Alerts is built with React Native, Expo, and TypeScript, and it uses the TMDb API as its primary content source. The app supports show discovery, detailed show and season views, watchlist management, episode notifications, user preferences, offline-friendly caching, and optional cloud synchronization. It is designed as a companion app rather than a streaming platform.

3.1.3 Definitions, Acronyms, and Abbreviations

API Application Programming Interface.

AsyncStorage

Local key-value storage used to persist app data on the device.

DFD Data Flow Diagram.

Expo The framework and toolchain used to build and run the app.

Firebase

Cloud platform used for optional authentication and data synchronization.

FR Functional Requirement.

SRS Software Requirements Specification.

TMDB The Movie Database, the external service used for show data.

UI User Interface.

3.1.4 References

A consolidated list of the external documents and sources used in this report is provided in the References chapter at the end of the document.

3.1.5 Overview

This SRS is organized into three main parts. The introduction explains the purpose and scope of the product, the overall description summarizes the product context and main characteristics, and the specific requirements section covers the external interfaces, functional behavior, performance expectations, constraints, and quality attributes.

3.2 Overall Description

3.2.1 Product Perspective

Episode Alerts is a single-user mobile app with a modular, service-oriented architecture. Navigation is handled through Expo Router, with tab-based screens for Home, Search, Watchlist, and Settings, while modal and detail screens support deeper navigation flows. The app depends on TMDB for content data, AsyncStorage for local persistence, and optional Firebase services for cloud synchronization.

3.2.2 Product Function

At a high level, the product provides the following functions:

- Discover popular, top-rated, and currently airing television shows.
- Search for shows by title and view detailed information.
- Display season and episode data, including next and last episode details.
- Let users add, remove, and manage shows in a watchlist.
- Notify users about upcoming episodes and release events.
- Store user preferences, analytics settings, and cached content.
- Synchronize selected user data to the cloud when enabled.

3.2.3 User Characteristics

The primary users are everyday TV viewers who want a simple way to follow multiple shows without manually tracking air dates. Most users are expected to have little or no technical background, so the app needs to stay easy to navigate, visually clear, and responsive. More advanced users may rely more heavily on search, filters, sorting, and settings.

3.2.4 General Constraints

The system is constrained by the Expo and React Native runtime, the availability of the TMDB API, and platform permission requirements for notifications and, where enabled, cloud features. The app must run on Android, iOS, and web with shared code, and it must keep sensitive values such as API keys outside the source tree. Offline behavior is limited to locally cached data and stored user content.

3.2.5 Assumptions and Dependencies

The system assumes that TMDB remains available and that the user has network access whenever fresh show data is needed. It also assumes that device permissions can be granted for notifications and that local storage is available for caching and preferences. Optional Firebase features depend on valid configuration and a signed-in user account.

3.3 Specific Requirements

3.3.1 External Interface Requirements

3.3.1.1 User Interfaces

The user interface shall provide a clean tab-based experience with Home, Search, Watchlist, and Settings screens. Detail pages shall present show metadata, season information, episode lists, and countdown information in a readable layout. The interface shall support light, dark, and system themes, show loading and error states, and remain usable across different screen sizes.

3.3.1.2 Hardware Interfaces

The application shall support touch input on mobile devices and pointer or keyboard input on web. It shall use device storage for local persistence and, when notifications are enabled, the operating system notification subsystem. If calendar or device-specific

sharing features are added in future updates, the app shall request the relevant platform permissions before access.

3.3.1.3 Software Interfaces

The application shall integrate with the TMDB API for all show, season, and episode data. It shall use AsyncStorage for local caching and user preference storage, Expo Notifications for reminders, Expo Network for connectivity awareness, and optional Firebase authentication and Firestore for cloud synchronization. The app shall also work with React Navigation and Expo Router for screen transitions.

3.3.1.4 Communication Interfaces

All external data exchange shall use secure HTTPS requests. The app shall communicate with TMDB through REST endpoints and shall use Firebase network services only when cloud sync is configured and the user is signed in. Notification interactions shall be handled through the platform notification response channel so tapping a reminder can open the related show.

3.3.2 Functional Requirements

3.3.2.1 Model 1: Discovery and Search

- FR-1.1** The system shall load and display popular, top-rated, and currently airing TV shows from TMDB.
- FR-1.2** The system shall allow the user to search for TV shows by title or keyword.
- FR-1.3** The system shall present a featured show on the home screen and allow the user to refresh the content.
- FR-1.4** The system shall use cached data when live data is unavailable so the user can still view previously loaded content.

3.3.2.2 Model 2: Show Details and Seasons

- FR-2.1** The system shall display show metadata such as overview, rating, status, genres, and network information.
- FR-2.2** The system shall display season information and the episodes available for each season.
- FR-2.3** The system shall show next episode and last episode details when the information is available from TMDB.

FR-2.4 The system shall present episode countdown and episode details in a dedicated view.

3.3.2.3 Model 3: Watchlist Management

FR-3.1 The system shall allow the user to add a show to the watchlist from discovery or detail screens.

FR-3.2 The system shall allow the user to remove a show from the watchlist.

FR-3.3 The system shall persist the watchlist locally so it is available after the app restarts.

FR-3.4 The system shall preserve watch progress and watch history for shows in the watchlist.

FR-3.5 The system shall support sorting and filtering of watchlist content where applicable in the user interface.

3.3.2.4 Model 4: Notifications and Preferences

FR-4.1 The system shall let the user enable or disable episode notifications.

FR-4.2 The system shall request notification permissions before scheduling reminders.

FR-4.3 The system shall schedule notifications for upcoming episodes when notifications are enabled.

FR-4.4 The system shall allow the user to change theme, analytics, and notification preferences.

FR-4.5 The system shall open the relevant show when the user taps a notification.

3.3.2.5 Model 5: Synchronization and Analytics

FR-5.1 The system shall support optional cloud synchronization of watchlist and preference data.

FR-5.2 The system shall synchronize local changes to the cloud when the user is signed in and cloud sync is available.

FR-5.3 The system shall record screen views and selected user actions when analytics are enabled.

FR-5.4 The system shall respect the user's analytics preference and stop collecting events when analytics are disabled.

3.3.3 Performance Requirements

The application should remain responsive while data is loaded or synchronized in the background. Cached content should appear quickly when available, and common actions such as adding or removing a show from the watchlist should finish without noticeable delay. Notification synchronization and cloud synchronization should not block the main user interface.

3.3.4 Design Constraints

The implementation shall remain within the React Native, Expo, and TypeScript stack already used by the project. The system shall rely on TMDB as the primary content source and shall not require direct changes to third-party services. Local persistence shall continue to use AsyncStorage, and cloud synchronization shall remain optional so the app can still work without Firebase configuration.

3.3.5 Attributes

The system shall prioritize usability, reliability, portability, maintainability, and security. The interface should be easy to understand for non-technical users, data should persist consistently across sessions, and the codebase should stay modular so new features can be added without major restructuring.

3.3.6 Other Attributes

The application should support offline use where cached data is available, preserve user privacy by keeping analytics opt-in, and remain practical for everyday mobile use. Future work may extend support for more detailed personalization, additional calendar integration, or broader cloud features without changing the core product goals.

Design

4.1 UML

4.1.1 Sequence Diagram

Figure 4.1 illustrates the UML sequence diagram representing the interaction between different components of the system, including user actions, application services, caching mechanisms, and external APIs.

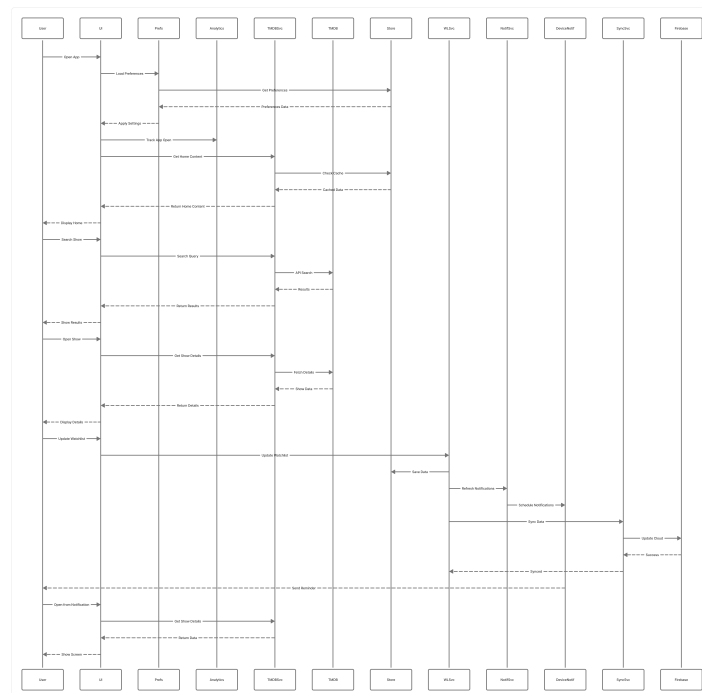


Figure 4.1: UML Sequence Diagram of the System Workflow

4.2 DFD

Episode Alerts is built around a few core data paths: show discovery and search through TMDB, watchlist and preference storage on the device, episode reminders through the notification layer, and optional cloud sync through Firebase.

4.2.1 Level 0 Context DFD

This level shows Episode Alerts as a single system interacting with the user and its external services.

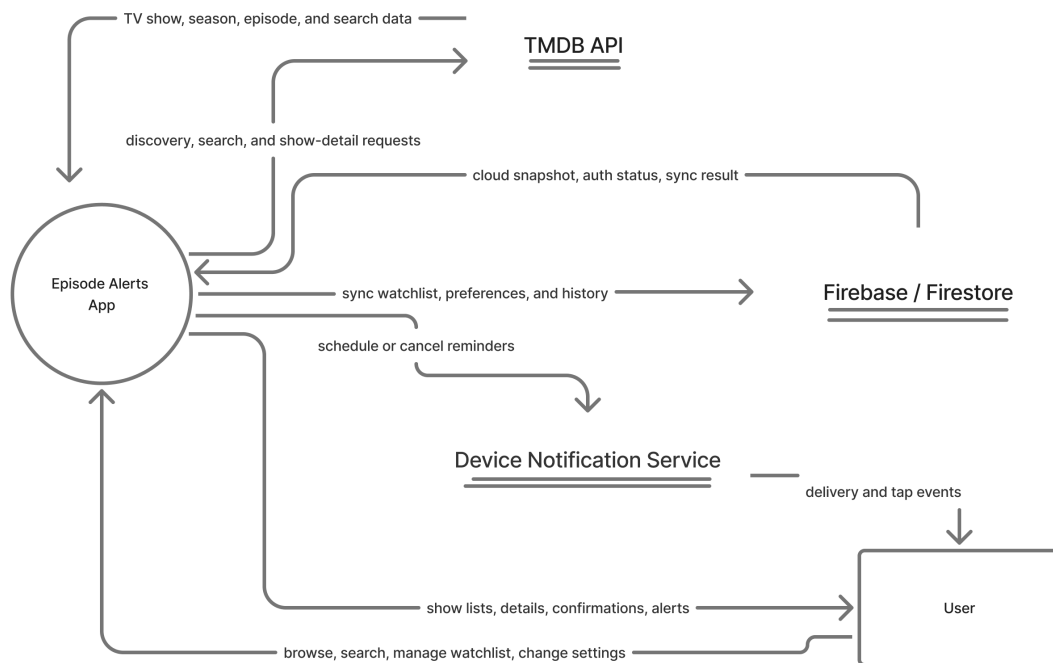


Figure 4.2: Level 0 context DFD for Episode Alerts.

4.2.2 Level 1 Logical DFD

This level breaks the app into its main logical processes and the data stores they use.

4.2.3 Level 2 DFD: Content Discovery and Search

This level decomposes the content discovery path because it is the main entry point into the app and the clearest place to show TMDBService, cache hydration, request normalization, and stale-data fallback. It is intentionally more detailed than level 1.

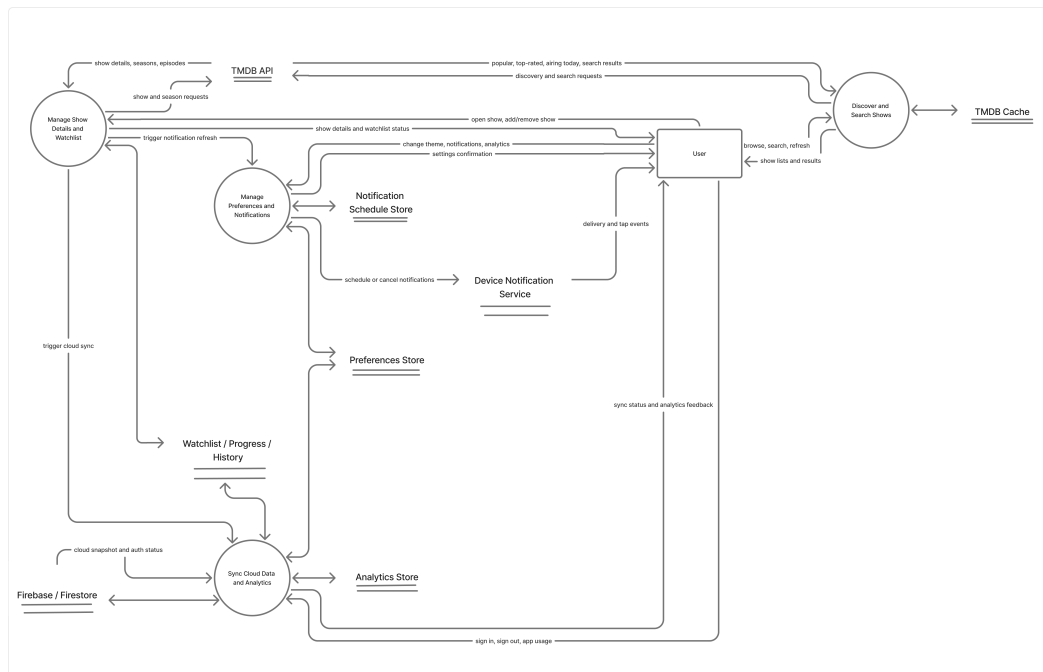


Figure 4.3: Level 1 logical DFD showing the primary processes and data stores.

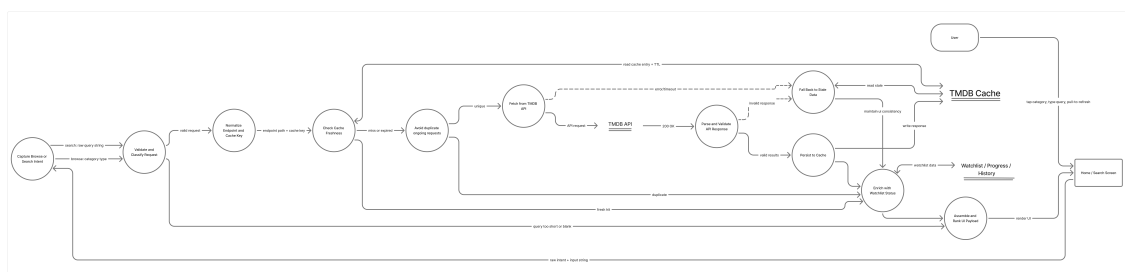


Figure 4.4: Level 2 DFD for content discovery and search, including cache and stale-data fallback paths.

4.3 Use Cases

System

Episode Alerts is a mobile companion app for discovering and tracking television shows.

Assumptions

- TMDB is available when fresh show data is requested, or cached data is already stored on the device.
- The user has granted notification permissions if episode reminders are enabled.
- Firebase is configured only when optional cloud synchronization is being used.
- Local device storage is available for watchlist data, preferences, and cached content.

The Main Use Cases Are

- Discover TV Shows
- Search for TV Shows
- View Show Details
- Manage Watchlist
- Configure Preferences and Notifications
- Sync Data to Cloud

1. Use Case 1: Discover TV Shows

1.1 Primary Actor: User

1.2 Precondition: The app is open and the Home screen is available.

1.3 Scenario: The user browses the Home screen to explore featured and trending content.

1.3.1 Main Success Scenario

1. The user opens the Home screen.
2. The system loads featured, airing today, popular, and top-rated shows from TMDB or from cached data.
3. The system displays the show sections in a scrollable layout.
4. The user taps a show card.
5. The system opens the show details screen for the selected show.

1.3.2 Exception Scenario

- **2a.** If step 2 fails because the network is unavailable, the system shows stale cached data and a stale-data indicator.
- **4a.** If step 4 cannot be completed because the selected item is unavailable, the system stays on the Home screen.

2. Use Case 2: Search for TV Shows

2.1 Primary Actor: User

2.2 Precondition: The Search screen is available and the user can enter text.

2.3 Scenario: The user enters a query to find a specific TV show.

2.3.1 Main Success Scenario

1. The user opens the Search screen.
2. The user types a show title or keyword.
3. The system waits for a valid query and sends the request to TMDB.
4. The system displays the matching search results.
5. The user selects one of the results.
6. The system opens the show details screen for that result.

2.3.2 Exception Scenario

- **3a.** If step 3 fails because the request is too short, the system clears the results and waits for a valid search term.
- **3b.** If the network request fails, the system shows an error message and allows the user to retry.
- **4a.** If no results are returned in step 4, the system shows an empty state instead of a result list.

3. Use Case 3: View Show Details

3.1 Primary Actor: User

3.2 Precondition: The user has selected a show from Home, Search, or Watchlist.

3.3 Scenario: The user opens a show to review detailed information and explore seasons or episodes.

3.3.1 Main Success Scenario

1. The user opens a show details screen.

2. The system loads show metadata such as overview, rating, status, genres, and network information.
3. The system displays next episode information, last episode information, and available seasons.
4. The user opens a season to view its episode list.
5. The user returns to the show details screen or continues to another related action, such as adding the show to the watchlist.

3.3.2 Exception Scenario

- **2a.** If step 2 fails, the system falls back to cached details when available.
- **3a.** If a show does not have a next episode yet, the countdown area is hidden instead of showing invalid information.
- **4a.** If season data cannot be loaded, the system shows an error or empty state for that section only.

4. Use Case 4: Manage Watchlist

4.1 Primary Actor: User

4.2 Precondition: The user is viewing a show card, show details screen, or the Watchlist screen.

4.3 Scenario: The user adds, removes, or organizes shows in the watchlist.

4.3.1 Main Success Scenario

1. The user taps the watchlist action for a show.
2. The system saves the show locally in the watchlist store.
3. The system updates the watchlist screen and related counts immediately.
4. The system schedules release notifications when notifications are enabled.
5. The user later removes the show or clears the watchlist if needed.

4.3.2 Exception Scenario

- **1a.** If step 1 targets a show that is already in the watchlist, the system leaves the list unchanged and returns a duplicate-state response.
- **2a.** If step 2 fails because storage is unavailable, the system shows an error and keeps the previous watchlist state.
- **5a.** If the user clears the watchlist and later chooses undo, the system restores the last saved snapshot when available.

5. Use Case 5: Configure Preferences and Notifications

5.1 Primary Actor: User

5.2 Precondition: The Settings screen is available.

5.3 Scenario: The user changes app preferences such as theme, analytics, and notification behavior.

5.3.1 Main Success Scenario

1. The user opens the Settings screen.
2. The user changes the theme, analytics preference, or notification setting.
3. The system saves the preference locally and updates the app behavior.
4. If notifications are enabled, the system requests permission and schedules upcoming episode reminders.
5. The updated setting remains active the next time the app is opened.

5.3.2 Exception Scenario

- **2a.** If a preference update cannot be saved, the system shows an error and retains the previous value.
- **3a.** If analytics are disabled, the system stops recording usage events immediately.
- **4a.** If notification permission is denied in step 4, the system keeps notifications disabled and does not schedule reminders.

6. Use Case 6: Sync Data to Cloud

6.1 Primary Actor: Signed-in User

6.2 Precondition: Firebase is configured and the user is signed in.

6.3 Scenario: The system synchronizes watchlist data and user preferences with the cloud.

6.3.1 Main Success Scenario

1. The user signs in or triggers a synchronization event.
2. The system builds a payload from the watchlist, watch progress, watch history, and selected preferences.
3. The system uploads the payload to Firestore.
4. The system stores the last successful sync time.
5. The system repeats synchronization automatically when local data changes or when the app resumes.

6.3.2 Exception Scenario

- **1a.** If step 1 occurs while the device is offline, the system skips the upload and retries later.
- **3a.** If Firebase is not configured or the user is not signed in, the system keeps all data local.
- **5a.** If the cloud request fails, the system preserves local data and logs the sync failure for later retry.

Chapter 5

Code

5.1 Implementation Overview

The codebase is centered around a small set of services that support the app's main flow: fetching TV data, managing the user's watchlist, scheduling notifications, and persisting preferences. For this report, the two most important modules are TMDbservice and WatchlistService because they drive the core data flow and the user's personalized tracking experience.

5.1.1 TMDbservice

TMDbservice is the app's main data layer for TV show content. It communicates with TMDB, normalizes show and episode data, and adds caching so the app can keep working even when the network is slow or unavailable. This service is central to the Home, Search, and Show Details screens because almost all show information comes through it.

```
1 import { TMDB_CONFIG, API_KEY } from '../constants/Config';
2 import AsyncStorage from '@react-native-async-storage/async-storage';
3 import { reportError } from '@app/utils/errorHandling';
4
5 export interface TVShow {
6   id: number;
7   name: string;
8   overview: string;
9   poster_path: string;
10  backdrop_path: string;
11  vote_average: number;
12  first_air_date: string;
13  next_episode_to_air?: Episode;
14  last_episode_to_air?: Episode;
```

```
15   number_of_seasons: number;
16   number_of_episodes?: number;
17   status: string;
18   genres: Genre[];
19   networks: Network[];
20   created_by: Creator[];
21   seasons?: Season[];
22   addedAt?: number;
23 }
24
25 export interface Episode {
26   id: number;
27   name: string;
28   overview: string;
29   still_path: string;
30   air_date: string;
31   episode_number: number;
32   season_number: number;
33   vote_average: number;
34   runtime?: number;
35 }
36
37 export interface Season {
38   id: number;
39   name: string;
40   overview: string;
41   poster_path: string;
42   air_date: string;
43   season_number: number;
44   episode_count: number;
45   episodes?: Episode[];
46 }
47
48 export interface Genre {
49   id: number;
50   name: string;
51 }
52
53 export interface Network {
54   id: number;
55   name: string;
```

```
56     logo_path: string;
57 }
58
59 export interface Creator {
60     id: number;
61     name: string;
62     profile_path: string;
63 }
64
65 export interface CastMember {
66     id: number;
67     name: string;
68     profile_path: string | null;
69     character?: string;
70     order?: number;
71 }
72
73 interface APIResponse<T> {
74     page?: number;
75     results: T[];
76     total_pages?: number;
77     total_results?: number;
78 }
79
80 interface CachedResponseEntry {
81     data: unknown;
82     expiresAt: number;
83     updatedAt: number;
84 }
85
86 interface TVShowCreditsResponse {
87     cast: CastMember[];
88 }
89
90 const API_CACHE_STORAGE_KEY = '@EpisodeAlerts:tmdbApiCache';
91 const DEFAULT_CACHE_TTL_MS = 15 * 60 * 1000;
92 const MAX_CACHE_ENTRIES = 200;
93 const PERSIST_DEBOUNCE_MS = 400;
94 const MAX_STALE_RETENTION_MS = 7 * 24 * 60 * 60 * 1000;
95
96 class TMDBService {
```

```
97 private static instance: TMDBService;
98 private baseUrl: string;
99 private headers: HeadersInit;
100 private memoryCache = new Map<string, CachedResponseEntry>();
101 private inFlightRequests = new Map<string, Promise<unknown>>();
102 private isCacheHydrated = false;
103 private hydratePromise: Promise<void> | null = null;
104 private persistTimer: ReturnType<typeof setTimeout> | null =
    null;
105 private staleFallbackUsed = false;
106 private lastCachedDataUpdatedAt: number | null = null;
107
108 private constructor() {
109     this.baseUrl = TMDB_CONFIG.BASE_URL;
110     this.headers = {
111         'Authorization': 'Bearer ${API_KEY}',
112         'Content-Type': 'application/json',
113     };
114 }
115
116 public static getInstance(): TMDBService {
117     if (!TMDBService.instance) {
118         TMDBService.instance = new TMDBService();
119     }
120     return TMDBService.instance;
121 }
122
123 private normalizeEndpoint(endpoint: string): string {
124     const [path, queryString] = endpoint.split('?');
125     if (!queryString) {
126         return path;
127     }
128
129     const params = new URLSearchParams(queryString);
130     const sortedEntries = Array.from(params.entries()).sort((a, b)
        => {
131         if (a[0] === b[0]) {
132             return a[1].localeCompare(b[1]);
133         }
134         return a[0].localeCompare(b[0]);
135     });
```

```
136
137     const normalizedParams = new URLSearchParams();
138     sortedEntries.forEach(([key, value]) => {
139         normalizedParams.append(key, value);
140     });
141
142     return `${path}?${normalizedParams.toString()}`;
143 }
144
145 private getCacheTTL(endpoint: string): number {
146     const normalized = this.normalizeEndpoint(endpoint);
147
148     if (normalized.startsWith('/search/')) {
149         return 5 * 60 * 1000;
150     }
151
152     if (normalized.startsWith('/tv/airing_today')) {
153         return 10 * 60 * 1000;
154     }
155
156     if (normalized.startsWith('/tv/popular') || normalized.
157         startsWith('/tv/top_rated')) {
158         return 30 * 60 * 1000;
159     }
160
161     if (/^\/tv\/\d+\/season\/\d+\/.test(normalized)) {
162         return 6 * 60 * 60 * 1000;
163     }
164
165     if (/^\/tv\/\d+\/.test(normalized)) {
166         return 60 * 60 * 1000;
167     }
168
169     return DEFAULT_CACHE_TTL_MS;
170 }
171
172 private buildCacheKey(endpoint: string): string {
173     return this.normalizeEndpoint(endpoint);
174 }
175
176 private async ensureCacheHydrated(): Promise<void> {
```



```
176     if (this.isCacheHydrated) {
177         return;
178     }
179
180     if (this.hydratePromise) {
181         await this.hydratePromise;
182         return;
183     }
184
185     this.hydratePromise = (async () => {
186         try {
187             const raw = await AsyncStorage.getItem(
188                 API_CACHE_STORAGE_KEY);
189             if (!raw) {
190                 this.isCacheHydrated = true;
191                 return;
192             }
193
194             const parsed = JSON.parse(raw) as Record<string,
195                 CachedResponseEntry>;
196             const now = Date.now();
197             Object.entries(parsed).forEach(([key, entry]) => {
198                 if (!entry || typeof entry.expiresAt !== 'number') {
199                     return;
200                 }
201
202                 const isFresh = entry.expiresAt > now;
203                 const isWithinStaleRetention = now - entry.expiresAt <=
204                     MAX_STALE_RETENTION_MS;
205
206                 if (isFresh || isWithinStaleRetention) {
207                     this.memoryCache.set(key, entry);
208                 }
209             });
210             this.isCacheHydrated = true;
211         } catch (error) {
212             reportError('TMDBService.ensureCacheHydrated', error, {
213                 fallbackMessage: 'Failed to hydrate cached API data.',
214                 trackAnalytics: false,
215             });
216             this.isCacheHydrated = true;
217         }
218     });
```

```
214     } finally {
215         this.hydratePromise = null;
216     }
217     })();
218
219     await this.hydratePromise;
220 }
221
222 private getCacheResponse<T>(cacheKey: string): { data: T | null
223     ; isStale: boolean; updatedAt: number | null } {
224     const entry = this.memoryCache.get(cacheKey);
225     if (!entry) {
226         return {
227             data: null,
228             isStale: false,
229             updatedAt: null,
230         };
231     }
232     if (entry.expiresAt <= Date.now()) {
233         return {
234             data: entry.data as T,
235             isStale: true,
236             updatedAt: entry.updatedAt,
237         };
238     }
239
240     return {
241         data: entry.data as T,
242         isStale: false,
243         updatedAt: entry.updatedAt,
244     };
245 }
246
247 private trimCacheIfNeeded(): void {
248     if (this.memoryCache.size <= MAX_CACHE_ENTRIES) {
249         return;
250     }
251
252     const oldestEntries = Array.from(this.memoryCache.entries())
253         .sort((a, b) => a[1].updatedAt - b[1].updatedAt)
```

```
254     .slice(0, this.memoryCache.size - MAX_CACHE_ENTRIES);
255
256     oldestEntries.forEach(([key]) => {
257         this.memoryCache.delete(key);
258     });
259 }
260
261 private setCachedResponse(cacheKey: string, data: unknown, ttlMs
    : number): void {
262     const now = Date.now();
263     this.memoryCache.set(cacheKey, {
264         data,
265         expiresAt: now + ttlMs,
266         updatedAt: now,
267     });
268
269     this.trimCacheIfNeeded();
270     this.scheduleCachePersist();
271 }
272
273 private scheduleCachePersist(): void {
274     if (this.persistTimer) {
275         clearTimeout(this.persistTimer);
276     }
277
278     this.persistTimer = setTimeout(() => {
279         this.persistTimer = null;
280         this.persistCache().catch((error) => {
281             reportError('TMDBService.persistCache', error, {
282                 fallbackMessage: 'Failed to persist cached API data.',
283                 trackAnalytics: false,
284             });
285         });
286     }, PERSIST_DEBOUNCE_MS);
287 }
288
289 private async persistCache(): Promise<void> {
290     const serialized = Object.fromEntries(this.memoryCache.entries
        ());
291     await AsyncStorage.setItem(API_CACHE_STORAGE_KEY, JSON.
        stringify(serialized));
```

```
292     }
293
294     public async clearCache(): Promise<void> {
295         this.memoryCache.clear();
296         this.inFlightRequests.clear();
297         this.lastCachedDataUpdatedAt = null;
298         this.staleFallbackUsed = false;
299         await AsyncStorage.removeItem(API_CACHE_STORAGE_KEY);
300     }
301
302     public consumeStaleFallbackFlag(): boolean {
303         const staleFallbackUsed = this.staleFallbackUsed;
304         this.staleFallbackUsed = false;
305         return staleFallbackUsed;
306     }
307
308     public getLastCachedDataUpdatedAt(): number | null {
309         return this.lastCachedDataUpdatedAt;
310     }
311
312     private async fetchAPI<T>(endpoint: string): Promise<T> {
313         await this.ensureCacheHydrated();
314
315         const cacheKey = this.buildCacheKey(endpoint);
316         const cachedResponse = this.getCachedResponse<T>(cacheKey);
317         if (cachedResponse.data && !cachedResponse.isStale) {
318             this.lastCachedDataUpdatedAt = cachedResponse.updatedAt;
319             return cachedResponse.data;
320         }
321
322         const existingRequest = this.inFlightRequests.get(cacheKey);
323         if (existingRequest) {
324             return existingRequest as Promise<T>;
325         }
326
327         const requestPromise = (async () => {
328             try {
329                 const url = `${this.baseURL}${endpoint}`;
330                 const response = await fetch(url, {
331                     method: 'GET',
332                     headers: this.headers,
```

```
333         });
334
335         if (!response.ok) {
336             const errorData = await response.json().catch(() => null
337             );
338             throw new Error('HTTP error! status: ${response.status},
339                 message: ${errorData?.status_message || 'Unknown
340                 error'}');
341         }
342
343         const data = (await response.json()) as T;
344         this.setCachedResponse(cacheKey, data, this.getCacheTTL(
345             endpoint));
346         this.lastCachedDataUpdatedAt = Date.now();
347         return data;
348     } catch (error) {
349         if (cachedResponse.data) {
350             this.staleFallbackUsed = true;
351             this.lastCachedDataUpdatedAt = cachedResponse.updatedAt;
352             return cachedResponse.data;
353         }
354
355         throw reportError('TMDBService.fetchAPI', error, {
356             fallbackMessage: 'Unable to load data from server.
357             Please try again.',
358         });
359     } finally {
360         this.inFlightRequests.delete(cacheKey);
361     }
362 })();
363
364 this.inFlightRequests.set(cacheKey, requestPromise);
365
366 try {
367     return await requestPromise;
368 } catch (error) {
369     throw error;
370 }
371
372 public async getPopularTVShows(page = 1): Promise<APIResponse <
```

```
TVShow>> {
369   return this.fetchAPI<APIResponse<TVShow>>('/tv/popular?page=${
        page}'));
370 }
371
372 public async getTVShowDetails(id: number): Promise<TVShow> {
373   return this.fetchAPI<TVShow>('/tv/${id}');
374 }
375
376 public async getTVShowCast(id: number): Promise<CastMember[]> {
377   const response = await this.fetchAPI<TVShowCreditsResponse>('/
        tv/${id}/credits');
378   return Array.isArray(response.cast) ? response.cast : [];
379 }
380
381 public async searchTVShows(query: string, page = 1): Promise<
        APIResponse<TVShow>> {
382   return this.fetchAPI<APIResponse<TVShow>>('/search/tv?query=${
        encodeURIComponent(query)}&page=${page}');
383 }
384
385 public async getTopRatedTVShows(page = 1): Promise<APIResponse<
        TVShow>> {
386   return this.fetchAPI<APIResponse<TVShow>>('/tv/top_rated?page=
        ${page}');
387 }
388
389 public async getTVShowsAiringToday(page = 1): Promise<
        APIResponse<TVShow>> {
390   return this.fetchAPI<APIResponse<TVShow>>('/tv/airing_today?
        page=${page}');
391 }
392
393 public async getSeasonDetails(tvId: number, seasonNumber: number
        ): Promise<Season> {
394   return this.fetchAPI<Season>('/tv/${tvId}/season/${
        seasonNumber}');
395 }
396
397 public getImageUrl(path: string, size: string): string {
398   return `${TMDB_CONFIG.IMAGE_BASE_URL}/${size}${path}`;
```

```
399     }
400   }
401
402   export default TMDBService.getInstance();
```

5.1.2 WatchlistService

WatchlistService manages the user's saved shows, watch progress, and watch history. It stores that data locally with AsyncStorage and can trigger cloud synchronization when Firebase is available. This service is important because it supports the app's personalized experience and connects the watchlist, notifications, and settings features.

```
1  import AsyncStorage from '@react-native-async-storage/async-storage';
2  import { TVShow, Episode } from './TMDBService';
3  import { reportError } from '@app/utils/errorHandling';
4  import { hasEpisodeAired } from '@app/utils/episodeRelease';
5
6  const WATCHLIST_STORAGE_KEY = '@EpisodeAlerts:watchlist';
7  const WATCH_PROGRESS_STORAGE_KEY = '@EpisodeAlerts:watchProgress';
8  const WATCH_HISTORY_STORAGE_KEY = '@EpisodeAlerts:watchHistory';
9
10 export interface LastWatchedEpisode {
11   showId: number;
12   showName: string;
13   seasonNumber: number;
14   episodeNumber: number;
15   episodeName: string;
16   watchedAt: number;
17 }
18
19 export interface WatchHistoryEntry extends LastWatchedEpisode {
20   id: string;
21 }
22
23 export interface WatchlistEntry {
24   show: TVShow;
25   addedAt: number;
26 }
27
28 export interface WatchlistSnapshot {
29   watchlist: WatchlistEntry[];
```

```
30 progressMap: Record<string, LastWatchedEpisode>;
31 historyMap: Record<string, WatchHistoryEntry[]>;
32 }
33
34 class WatchlistService {
35     private static instance: WatchlistService;
36
37     private constructor() {}
38
39     public static getInstance(): WatchlistService {
40         if (!WatchlistService.instance) {
41             WatchlistService.instance = new WatchlistService();
42         }
43         return WatchlistService.instance;
44     }
45
46     private triggerCloudSync(): void {
47         void import('./CloudSyncService')
48             .then((module) => module.default.syncToCloudIfSignedIn())
49             .catch((error) => {
50                 reportError('WatchlistService.triggerCloudSync', error, {
51                     fallbackMessage: 'Cloud sync could not be triggered.',
52                 });
53             });
54     }
55
56     private normalizeWatchlist(rawWatchlist: unknown):
57         WatchlistEntry[] {
58         if (!Array.isArray(rawWatchlist)) {
59             return [];
60         }
61
62         const now = Date.now();
63         const normalized: WatchlistEntry[] = [];
64
65         rawWatchlist.forEach((item, index) => {
66             if (item && typeof item === 'object' && 'show' in item) {
67                 const entry = item as { show?: TVShow; addedAt?: number };
68                 if (!entry.show || typeof entry.show.id !== 'number') {
69                     return;
70                 }
71             }
72         });
73     }
74 }
```



```
70
71     normalized.push({
72         show: {
73             ...entry.show,
74             addedAt: entry.addedAt,
75         },
76         addedAt: typeof entry.addedAt === 'number' ? entry.
             addedAt : now,
77     });
78
79     return;
80 }
81
82 const show = item as TVShow;
83 if (!show || typeof show.id !== 'number') {
84     return;
85 }
86
87 const addedAt = show.addedAt ?? now - (rawWatchlist.length -
    index);
88 normalized.push({
89     show: {
90         ...show,
91         // Preserve original order for legacy arrays by creating
            stable timestamps.
92         addedAt,
93     },
94     addedAt,
95 });
96 });
97
98 return normalized;
99 }
100
101 private async getWatchlistEntries(): Promise<WatchlistEntry[]> {
102     try {
103         const watchlistJson = await AsyncStorage.getItem(
            WATCHLIST_STORAGE_KEY);
104         const normalized = this.normalizeWatchlist(watchlistJson ?
            JSON.parse(watchlistJson) : []);
105         await this.saveWatchlistEntries(normalized);
```

```
106     return normalized;
107   } catch (error) {
108     reportError('WatchlistService.getWatchlistEntries', error, {
109       fallbackMessage: 'Unable to load watchlist entries.',
110     });
111     return [];
112   }
113 }
114
115 private async saveWatchlistEntries(entries: WatchlistEntry[]):
116   Promise<void> {
117     await AsyncStorage.setItem(WATCHLIST_STORAGE_KEY, JSON.
118       stringify(entries));
119   }
120
121 async getWatchlist(): Promise<TVShow[]> {
122   try {
123     const entries = await this.getWatchlistEntries();
124     return entries.map((entry) => ({
125       ...entry.show,
126       addedAt: entry.addedAt,
127     }));
128   } catch (error) {
129     reportError('WatchlistService.getWatchlist', error, {
130       fallbackMessage: 'Unable to load your watchlist.',
131     });
132     return [];
133   }
134 }
135
136 async getWatchlistSnapshot(): Promise<WatchlistSnapshot> {
137   const [watchlist, progressMap, historyMap] = await Promise.all
138     ([
139       this.getWatchlistEntries(),
140       this.getProgressMap(),
141       this.getHistoryMap(),
142     ]);
143
144   return {
145     watchlist,
146     progressMap,
```

```
144     historyMap,
145   };
146 }
147
148 async restoreWatchlistSnapshot(
149   snapshot: WatchlistSnapshot,
150   options?: { skipCloudSync?: boolean },
151 ): Promise<boolean> {
152   try {
153     await this.saveWatchlistEntries(snapshot.watchlist);
154     await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
155       stringify(snapshot.progressMap));
156     await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.
157       stringify(snapshot.historyMap));
158
159     if (!options?.skipCloudSync) {
160       this.triggerCloudSync();
161     }
162
163     return true;
164   } catch (error) {
165     reportError('WatchlistService.restoreWatchlistSnapshot',
166       error, {
167         fallbackMessage: 'Unable to restore watchlist snapshot.',
168       });
169     return false;
170   }
171 }
172
173 async addToWatchlist(show: TVShow): Promise<boolean> {
174   try {
175     const currentEntries = await this.getWatchlistEntries();
176
177     if (currentEntries.some((item) => item.show.id === show.id))
178     {
179       return false;
180     }
181
182     const addedAt = Date.now();
183     const entry: WatchlistEntry = {
184       show: {
```

```
181         ...show,
182         addedAt,
183     },
184     addedAt,
185 };
186
187     const updatedWatchlist = [...currentEntries, entry];
188     await this.saveWatchlistEntries(updatedWatchlist);
189     this.triggerCloudSync();
190     return true;
191 } catch (error) {
192     reportError('WatchlistService.addToWatchlist', error, {
193         fallbackMessage: 'Unable to add this show to your
194         watchlist.',
195     });
196     return false;
197 }
198
199 async removeFromWatchlist(showId: number): Promise<boolean> {
200     try {
201         const currentWatchlist = await this.getWatchlistEntries();
202         const updatedWatchlist = currentWatchlist.filter((show) =>
203             show.show.id !== showId);
204
205         await this.saveWatchlistEntries(updatedWatchlist);
206         await this.clearLastWatchedEpisode(showId, { skipCloudSync:
207             true });
208         await this.clearWatchHistory(showId, { skipCloudSync: true
209             });
210         this.triggerCloudSync();
211         return true;
212     } catch (error) {
213         reportError('WatchlistService.removeFromWatchlist', error, {
214             fallbackMessage: 'Unable to remove this show from your
215             watchlist.',
216         });
217         return false;
218     }
219 }
```

```
217   async isInWatchlist(showId: number): Promise<boolean> {
218     try {
219       const watchlist = await this.getWatchlistEntries();
220       return watchlist.some((show) => show.show.id === showId);
221     } catch (error) {
222       reportError('WatchlistService.isInWatchlist', error, {
223         fallbackMessage: 'Unable to check watchlist status.',
224         trackAnalytics: false,
225       });
226       return false;
227     }
228   }
229
230   async clearWatchlist(): Promise<boolean> {
231     try {
232       await AsyncStorage.removeItem(WATCHLIST_STORAGE_KEY);
233       await AsyncStorage.removeItem(WATCH_PROGRESS_STORAGE_KEY);
234       await AsyncStorage.removeItem(WATCH_HISTORY_STORAGE_KEY);
235       this.triggerCloudSync();
236       return true;
237     } catch (error) {
238       reportError('WatchlistService.clearWatchlist', error, {
239         fallbackMessage: 'Unable to clear your watchlist.',
240       });
241       return false;
242     }
243   }
244
245   private async getProgressMap(): Promise<Record<string,
246     LastWatchedEpisode>> {
247     try {
248       const progressJson = await AsyncStorage.getItem(
249         WATCH_PROGRESS_STORAGE_KEY);
250       return progressJson ? JSON.parse(progressJson) : {};
251     } catch (error) {
252       reportError('WatchlistService.getProgressMap', error, {
253         fallbackMessage: 'Unable to load watch progress.',
254       });
255       return {};
```

```
256
257   async getLastWatchedEpisodes(): Promise<Record<string,
258       LastWatchedEpisode>> {
259       return this.getProgressMap();
260   }
261
262   private async getHistoryMap(): Promise<Record<string,
263       WatchHistoryEntry[]>> {
264       try {
265           const historyJson = await AsyncStorage.getItem(
266               WATCH_HISTORY_STORAGE_KEY);
267           return historyJson ? JSON.parse(historyJson) : {};
268       } catch (error) {
269           reportError('WatchlistService.getHistoryMap', error, {
270               fallbackMessage: 'Unable to load watch history.',
271           });
272           return {};
273       }
274   }
275
276   async getWatchHistory(showId: number): Promise<WatchHistoryEntry
277       []> {
278       const historyMap = await this.getHistoryMap();
279       return (historyMap[String(showId)] || []).sort((a, b) => b.
280           watchedAt - a.watchedAt);
281   }
282
283   private async appendWatchHistory(showId: number, showName:
284       string, episode: Episode): Promise<void> {
285       const historyMap = await this.getHistoryMap();
286       const showKey = String(showId);
287       const existing = historyMap[showKey] || [];
288       const nextEntry: WatchHistoryEntry = {
289           id: `${showId}-${episode.season_number}-${episode.
290               episode_number}-${Date.now()}`,
291           showId,
292           showName,
293           seasonNumber: episode.season_number,
294           episodeNumber: episode.episode_number,
295           episodeName: episode.name,
296           watchedAt: Date.now(),
```

```
290     };
291
292     const deduped = existing.filter(
293       (entry) =>
294         !(
295           entry.seasonNumber === episode.season_number &&
296           entry.episodeNumber === episode.episode_number &&
297           Math.abs(entry.watchedAt - nextEntry.watchedAt) < 60_000
298         ),
299     );
300
301     historyMap[showKey] = [nextEntry, ...deduped].slice(0, 50);
302     await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.
303       stringify(historyMap));
304   }
305
306   async getLastWatchedEpisode(showId: number): Promise<
307     LastWatchedEpisode | null> {
308     const progressMap = await this.getProgressMap();
309     return progressMap[String(showId)] || null;
310   }
311
312   async setLastWatchedEpisode(showId: number, showName: string,
313     episode: Episode): Promise<boolean> {
314     try {
315       if (!hasEpisodeAired(episode.air_date)) {
316         throw new Error('Cannot save progress for an unreleased
317           episode.');
```

```
327     await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
328         stringify(progressMap));
329     await this.appendWatchHistory(showId, showName, episode);
330     this.triggerCloudSync();
331     return true;
332 } catch (error) {
333     reportError('WatchlistService.setLastWatchedEpisode', error,
334         {
335             fallbackMessage: 'Unable to save watch progress.',
336         });
337     return false;
338 }
339
340 async clearLastWatchedEpisode(showId: number, options?: {
341     skipCloudSync?: boolean }): Promise<boolean> {
342     try {
343         const progressMap = await this.getProgressMap();
344         delete progressMap[String(showId)];
345         await AsyncStorage.setItem(WATCH_PROGRESS_STORAGE_KEY, JSON.
346             stringify(progressMap));
347
348         if (!options?.skipCloudSync) {
349             this.triggerCloudSync();
350         }
351
352         return true;
353     } catch (error) {
354         reportError('WatchlistService.clearLastWatchedEpisode',
355             error, {
356                 fallbackMessage: 'Unable to clear watch progress.',
357             });
358         return false;
359     }
360 }
361
362 async clearWatchHistory(showId: number, options?: {
363     skipCloudSync?: boolean }): Promise<boolean> {
364     try {
365         const historyMap = await this.getHistoryMap();
366         delete historyMap[String(showId)];
```



```
362     await AsyncStorage.setItem(WATCH_HISTORY_STORAGE_KEY, JSON.  
363         stringify(historyMap));  
364  
365     if (!options?.skipCloudSync) {  
366         this.triggerCloudSync();  
367     }  
368  
369     return true;  
370 } catch (error) {  
371     reportError('WatchlistService.clearWatchHistory', error, {  
372         fallbackMessage: 'Unable to clear watch history.',  
373     });  
374     return false;  
375 }  
376 }  
377  
378 export default WatchlistService.getInstance();
```

Chapter 6

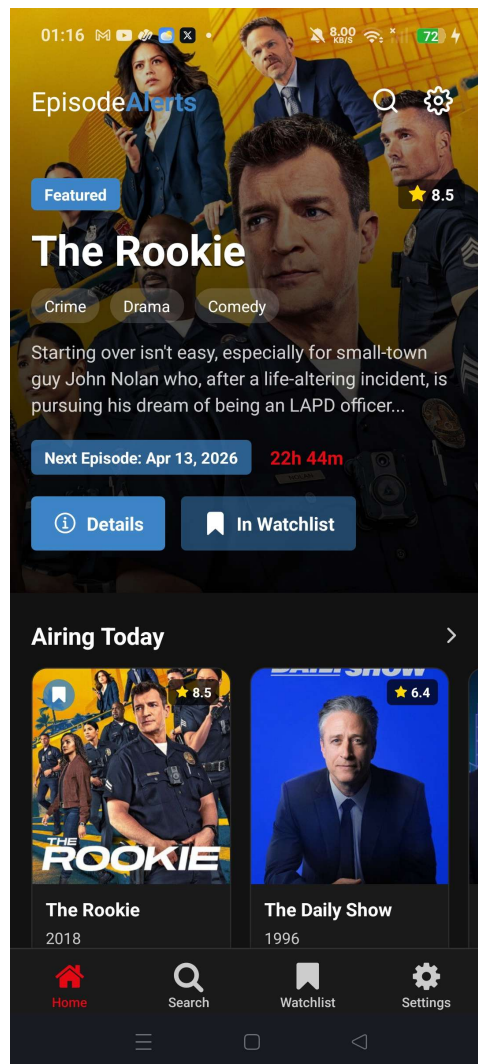
Results

6.1 Results Overview

The screenshots below show the implemented app in its final form, covering the main user flows: discovery, show details, watchlist management, analytics, and settings. Together, they demonstrate that the core screens are working as intended and that the app presents a consistent experience across its main features.

Source code is available on GitHub at [EpisodeAlerts-MobileApp](#).

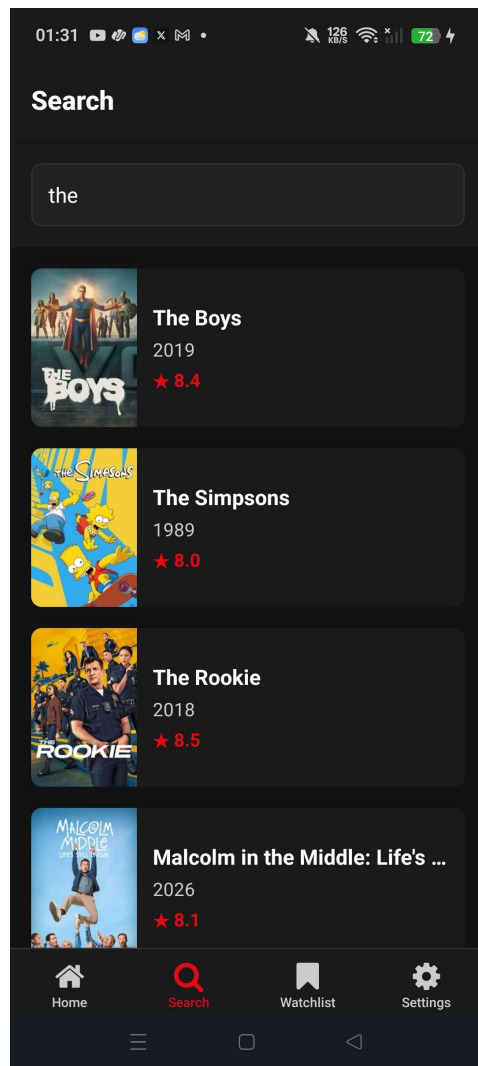
6.2 Application Screenshots



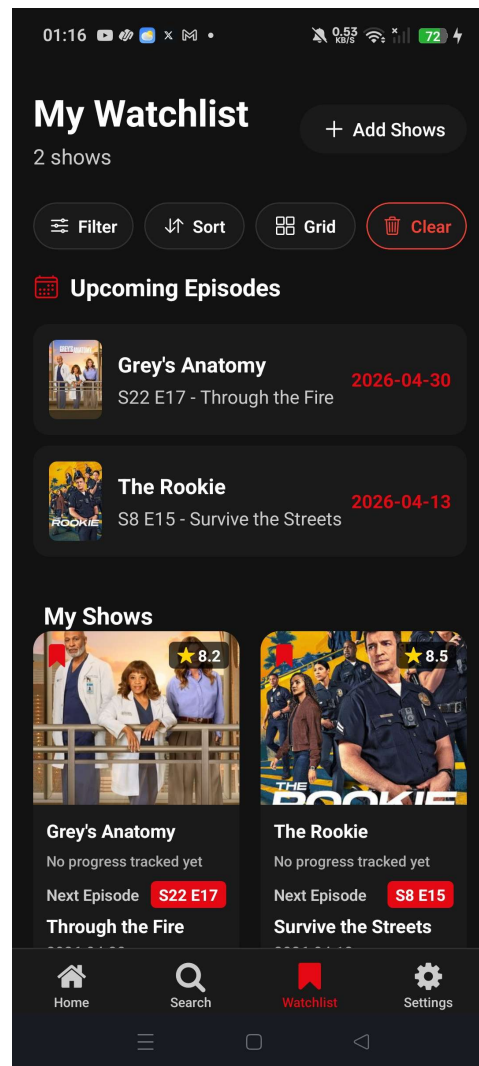
Home screen with featured content and category sections.



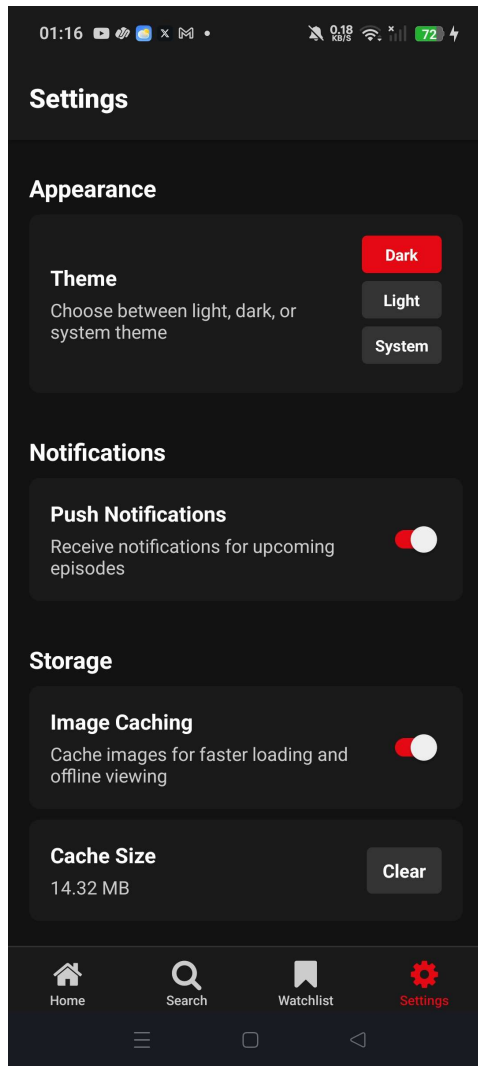
Show details screen with episode information and watchlist state.



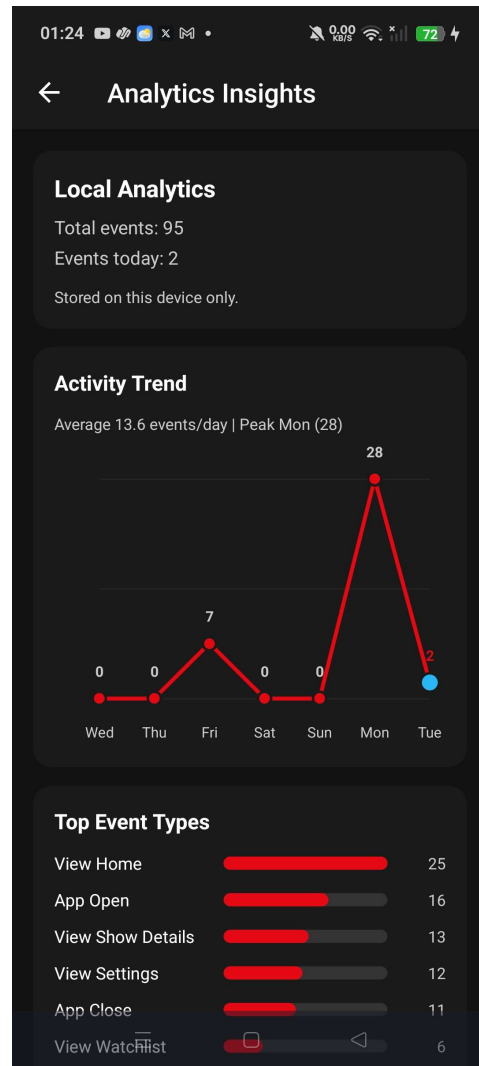
Search screen showing TV show search results.



Watchlist screen with upcoming episodes and saved shows.



Settings screen with theme, notifications, and caching options.



Analytics insights screen showing usage trends and event breakdowns.

6.3 Outcome Summary

The implemented build satisfies the core functional objectives defined in the SRS and demonstrates stable behavior across the primary user journeys.

- Discovery and search workflows successfully fetch and display TV data from TMDB, while preserving usability through cache and fallback behavior.
- Show details, season navigation, and next-episode context are presented consistently from Home, Search, and Watchlist entry points.
- Watchlist actions (add, remove, and update) persist correctly in local storage and reflect immediately in the user interface.
- Settings changes for theme, analytics, and notifications are retained and applied across app sessions.
- The analytics screen confirms event tracking visibility, and optional cloud-sync be-

havior aligns with the configured authentication state.

6.4 Conclusion

Episode Alerts delivers a complete and coherent mobile experience for TV discovery and tracking, with a clear architecture and reliable end-to-end flows. The final implementation is suitable for submission and provides a strong baseline for future enhancements such as richer personalization, expanded recommendation logic, and broader sync capabilities.

References

Documentation

- [1] **The Movie Database (TMDB) API documentation.**
[TMDB docs](#). Accessed 16 Apr. 2026.
- [2] **Expo documentation.**
[Expo docs](#). Accessed 16 Apr. 2026.
- [3] **React Native documentation.**
[React Native docs](#). Accessed 16 Apr. 2026.
- [4] **Firebase documentation.**
[Firebase docs](#). Accessed 16 Apr. 2026.
- [5] **Episode Alerts project repository.**
[Project repository](#). Accessed 16 Apr. 2026.

Related Work and Context

- [1] **CineSync Tracker.**
App Store listing for a movie and TV watchlist app with discovery and release-tracking features.